4회차 · CDC / Iceberg 프로젝트 스터디

변경 이력에서 현재 상태로

Spark SQL · 중복 제거 · 버전 조건 MERGE

공통 예제의 규칙을 내 테이블 설계로 옮깁니다

흐름을 먼저 알고 설계한다

8회 단계적 진행 로드맵

STEP 1 — 1회차

개념 & 환경 구성

[핵심 학습]

- OT · 왜 CDC+Lakehouse인가
- CDC/Iceberg 내부 구조와 장단점
- 실습 환경 구성 및 공통 예제 흐름 체험

[프로젝트 진행 방식]

공통 예제 (orders·payments·refunds)로 전체 구조 파악

STEP 2 — 2~3회차

수집·적재 흐름 체득

[핵심 학습]

- 2회차: PostgreSQL → Debezium → Kafka 변경 수집
- 3회차: Flink → Iceberg Bronze 적재·지연 측정

[프로젝트 진행 방식]

공통 예제로 c/u/d·체크포인트·커밋을 직접 다룬 뒤, 3회차에 내 개인 프로젝트 도메인·테이블 설계 확정

STEP 3 — 4~6회차

내 프로젝트 본격 구현

[핵심 학습]

- 4회차: Spark SQL Silver 중복 제거·MERGE
- 5회차: dbt Gold 마트·품질 테스트
- 6회차: 과거 데이터 수정·삭제 소급 정정

[프로젝트 진행 방식]

내가 설계한 도메인 테이블과 지표를 파이프라인에 실제 투입·구현

STEP 4 — 7~8회차

성능 튜닝 & 운영 점검

[핵심 학습]

- 7회차: Iceberg 작은 파일 (rewrite_data_files)·파티션·쿼리 튜닝
- 8회차: Airflow 연동·장애 복구 점검·최종 리뷰

[프로젝트 진행 방식]

Iceberg의 단점(Small Files)을 직접 해결하는 전후 성능 비교 벤치마크 완성 (발표 별도)

같은 환불의 원천과 Bronze

환불 변경	같은 refund_id의 이력	변경 후 원천
생성 c	after.amount = 20000	현재 환불 1행
정정 u	20000 → 30000	현재 환불 1행
철회 d	before.amount = 30000	현재 환불 0행
Bronze	c / u / d 세 이력 보존	주문·결제는 유지

공통 의미 복습 · 결제 100000원, 환불 한 건의 생성·정정·철회

환불 이력 세 줄은 환불 세 건이 아닙니다

이력의 질문

무슨 변경이 있었나요?
같은 키의 여러 버전 보관

현재 상태의 질문

지금 유효한 환불은?
키·버전·삭제 규칙 필요

20000원과 30000원은 같은 환불의 서로 다른 버전입니다

오늘 확인할 다섯 가지 결과

검증할 것	기대 상태
PK 한 행	같은 키는 현재 snapshot에 한 행 보존
최신 버전	비교 범위 안에서 최신 원천 버전 선택
삭제 유지	늦은 과거 UPDATE가 삭제를 바꾸지 않기
같은 입력 재실행	업무 결과 유지 · 물리 쓰기와 구분
무효 입력 거절	검증 위반이면 MERGE 전에 중단

개인 원천 DDL을 한 테이블의 Silver 계약으로 연결합니다

Bronze에서 Silver로 갱신하는 경로

지난 회차의 수집 흐름

PostgreSQL
서비스 원천
주문·결제·환불

→

Debezium + Kafka
변경 이벤트
보관

→

Flink
지속 처리
원문 append

→

Iceberg Bronze
변경 이력
append 보관

오늘: Spark SQL로 Silver 갱신

→

Spark SQL
정규화
최신 후보
버전 MERGE

→

Iceberg Silver
엔티티 한 행
삭제·버전 보존

후속

→

Gold
dbt
다음 회차

이번 실험의 입력

Bronze 규격의 고정 fixture를 전용 영역에 준비
실제 CDC 수집·기존 공통 테이블 갱신과 구분

실행 역할

Python runner: SQL 제출·순서·결과 확인
Spark: 분산 처리·파일 쓰기·commit

오늘은 Bronze 규격 고정 입력으로 전용 Silver 규칙을 검증합니다

Silver 한 행의 의미와 키

현재 상태 한 행	엔티티 키
주문 한 건의 현재 상태	orders.order_id
결제 한 건의 현재 상태	payments.payment_id
환불 한 건의 현재 상태	refunds.refund_id
같은 결제에 여러 환불	payment_id로 환불을 하나로 줄이지 않기

Silver의 한 행 의미와 엔티티 키를 먼저 맞춥니다

중복 세 가지와 충돌을 구분합니다

유형	구분할 근거	오늘 규칙의 범위
전달 중복	같은 topic / partition / offset	같은 전달 레코드의 반복
변경 재전달	같은 원천 버전, 다른 offset	PK별 현재 상태 한 행
업무 중복	서로 다른 PK의 유사 업무	별도 업무 정책 필요
전달 위치 충돌	같은 topic·partition·offset	다른 원문이면 실행 중단

같은 금액이라는 이유만으로 서로 다른 환불을 중복으로 보지 않습니다

최신 버전의 기준

먼저 원천 버전

source.lsn을
숫자로 비교

같은 LSN일 때

같은 토픽·파티션의
kafka_offset 비교

이번 예제 범위

단일 원천
토픽별 1파티션

다른 원천·다른 파티션의 offset을 전역 순서로 비교하지 않습니다

LSN은 문자열이 아니라 숫자로 비교합니다

문자열 비교

문자열 내림차순은
'9'가 '10'보다 앞

→

숫자 비교

BIGINT 10이
BIGINT 9보다 큽니다

→

변환 실패

기본 CAST는 NULL 반환
실행기가 MERGE 전 중단

```
CAST(get_json_object(payload, '$.source.lsn')
    AS BIGINT) AS source_lsn
```

타입이 바뀌면 최신 후보의 순서도 달라질 수 있습니다

변경 종류에 따라 행을 고릅니다

```
{"op":"u",
 "before":{"refund_id":"refund-r1","amount":20000},
 "after":{"refund_id":"refund-r1","amount":30000},
 "source":{"lsn":200}}

{"op":"d",
 "before":{"refund_id":"refund-r1","amount":30000},
 "after":null,"source":{"lsn":300}}
```

고정 입력 축약 · c/u/r은 after, d는 before · tombstone은 별개

정규화와 유효 입력

정규화한 컬럼	타입	확인할 조건
refund_id	STRING	NULL PK이면 MERGE 중단
amount / source_lsn	BIGINT	NULL·변환 실패이면 중단
op / 전달 위치	c/u/r/d · 같은 토픽/파티션	알 수 없는 op·충돌이면 중단
is_deleted	BOOLEAN	검증된 op = d이면 true

runner가 상한 필터 전 전체 fixture를 검증합니다 · 파싱 SELECT 자체는 입력 검증이 아닙니다

같은 키의 후보에 순위를 매깁니다

같은 refund_id	버전·상태	후보 순위
생성 c	LSN 100 / 20000 / false	rn = 3
정정 u	LSN 200 / 30000 / false	rn = 2
철회 d	LSN 300 / 30000 / true	rn = 1
선택 결과	마지막 삭제 후보 한 행	이후 유효 상태 필터

삭제도 순위에 포함합니다 · 먼저 제외하면 초기 구축에서 삭제가 빠집니다

ROW_NUMBER로 최신 후보를 고릅니다

```
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY refund_id
    ORDER BY source_lsn DESC, kafka_offset DESC
  ) AS rn
  FROM typed
)
SELECT refund_id, amount, is_deleted, source_lsn, kafka_offset
FROM ranked WHERE rn = 1
```

PARTITION BY는 엔티티 묶음 · ORDER BY는 버전 우선순위

DISTINCT와 MAX(amount)의 한계

DISTINCT

다른 버전이면
여러 행이 남습니다

MAX(amount)

30000 → 10000 정정
과거 30000을 선택

버전으로 선택

최신 버전의
현재 금액은 10000

가상 반례 30000 → 10000 · 공통 fixture는 20000 → 30000입니다

MERGE의 세 가지 판단

```
MERGE INTO {silver} AS t
USING {incoming} AS s
ON t.refund_id = s.refund_id
WHEN MATCHED AND ( /* 다음 장의 버전 조건 */ )
THEN UPDATE SET /* 키 이외의 값·삭제·버전 컬럼 */
WHEN NOT MATCHED THEN INSERT ( /* 명시한 컬럼 */ )
VALUES ( /* 대응하는 s 컬럼 */ )
```

MERGE 구조 요약 · 실행할 전체 SQL은 sql/02-merge.sql에 있습니다

버전 가드와 늦은 변경

```
WHEN MATCHED AND (
  s.source_lsn > t.source_lsn
  OR (s.source_lsn = t.source_lsn
      AND s.kafka_offset > t.kafka_offset)
)
THEN UPDATE SET ...

-- 대상: 삭제 LSN 300 / 입력: 과거 UPDATE LSN 200, offset 999
-- 이 입력은 대상을 갱신할까요?
```

ROW_NUMBER는 입력 배치 안의 최신 · MERGE 가드는 기존 target과 비교

삭제 표시와 재처리

현재 snapshot의 보존 행

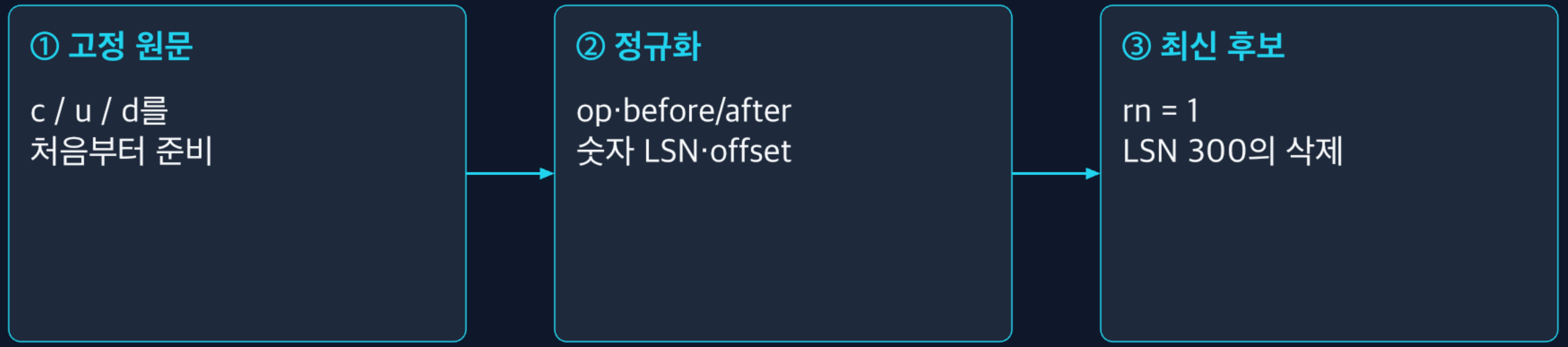
refund_id 한 행
amount = 30000
is_deleted = true
source_lsn = 300

현재 유효한 환불

WHERE NOT is_deleted
조회 결과 0행
과거 UPDATE로 부활 방지

삭제 행의 amount는 마지막 before 값이며 유효한 환불액이 아닙니다

시연 A — 고정 입력에서 후보 SELECT



시연 A · 고정 입력 전체의 최신 후보 확인, 다음은 상한별 단계 재생

시연 B — 생성과 정정을 차례로 반영

선택한 고정 입력 범위	현재 snapshot의 보존 행	확인할 값
c까지 · LSN ≤ 100	같은 refund_id 1행	20000 / false / LSN 100
u까지 · LSN ≤ 200	같은 refund_id 1행	30000 / false / LSN 200
해석	입력 범위를 선택한 실험	새 CDC 도착·운영 watermark 아님

시연 B · 전체 후보 d300 확인 후 c100부터 범위를 선택해 적용합니다

시연 B — 삭제를 현재 상태에 반영

현재 snapshot의 보존 행

COUNT(*) = 1
is_deleted = true
source_lsn = 300

현재 유효 행

WHERE NOT is_deleted
COUNT(*) = 0
격리 테이블만 변경

삭제 직전 값은 남기고, 현재 조회에서 삭제 표시 행을 제외합니다

시연 C — 같은 입력과 중복을 다시 적용

재적용 입력	업무 결과 기대	추가로 볼 것
동일 입력	PK 한 행 · 삭제 유지	10/26 기록: 새 snapshot·파일
위치 중복·23장	삭제 · LSN300 · offset12 유지	같은 topic·partition·offset
검증 근거	금액·is_deleted·PK 건수	논리 결과와 물리 쓰기 구분

시연 C · 업무 결과의 멱등성과 snapshot·파일 쓰기 비용을 나눠 봅니다

시연 C — 삭제 뒤 과거 UPDATE

① 과거 배치 선택

upper_lsn = 200
incoming u200
offset 13

② 대상과 비교

target d300
offset 12
LSN 우선 버전 가드

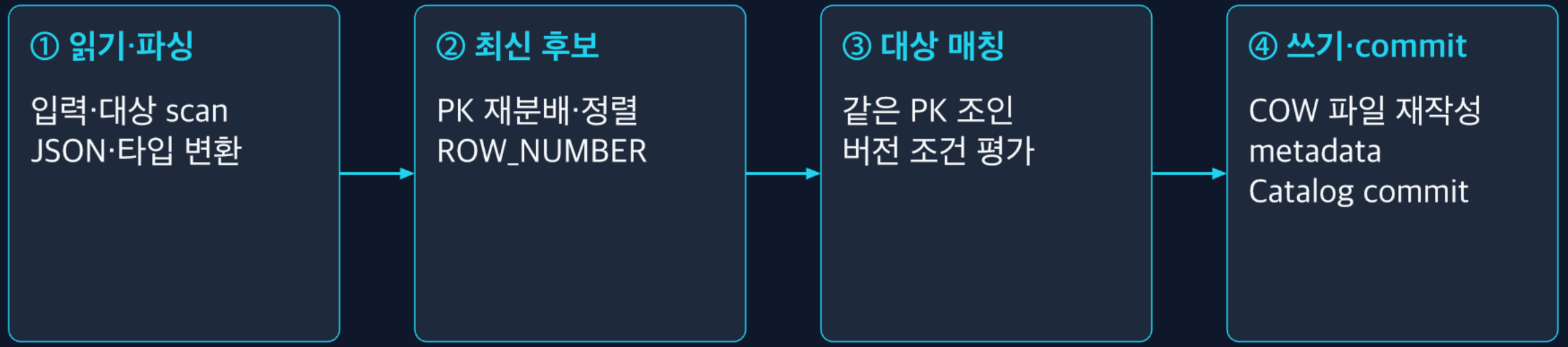
③ 삭제 상태 유지

guard_rejected = true
삭제 LSN 300 보존
현재 유효 환불 0행

시연 C · full scan의 후보 선택과 분리해, 과거 입력을 가드에 직접 넘깁니다

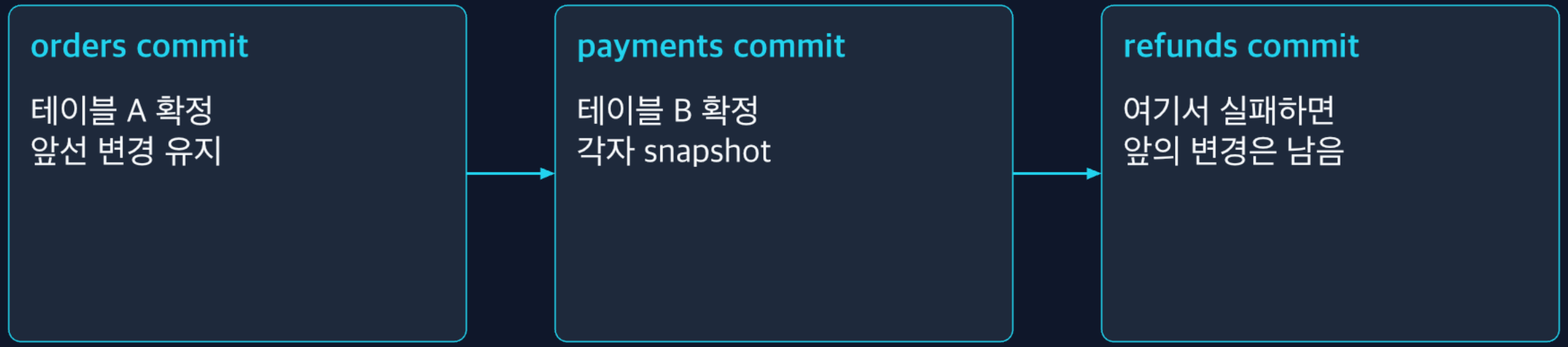
Spark SQL과 MERGE 내부 처리

Driver: 실행 조정 Executor: 데이터 처리
실제 물리 stage는 최적화·분배 상태에 따라 달라집니다



10/26 실행 관찰: 같은 입력 재실행에도 새 snapshot·파일이 생겼습니다

한 테이블의 commit과 세 테이블의 갱신



테이블별 상태·마지막 성공·재시도 범위와 후속 작업 조건을 관리합니다

전체 재조회와 증분 처리의 비용

갱신 방식	입력과 후보	함께 설계할 것
기준 구현	전체 Bronze scan → 최신 후보	읽기·정렬·재작성 증가
증분 구현	변경 범위 → 영향 키 → 후보	성공 위치·실패 재시도
늦은 변경·삭제	놓친 키·과거 범위 재계산	소급 정정은 6회차 연결

MERGE를 쓴다는 사실만으로 입력 SELECT가 증분이 되지는 않습니다

갱신 방식 선택과 운영 점검

선택지	맞는 조건	운영 부담
주기 MERGE	수십 분 주기·SQL 검증	매번 전체 scan·재작성
Spark 스트리밍	분 단위·같은 규칙·가드	상시 작업·commit·파일 정리
Flink upsert	더 짧은 지연 요구	상태 복구·삭제 파일 정리
관측할 것	입력·후보·무효행·결과	마지막 성공·파일·충돌

키·버전·삭제 규칙은 그대로, 실행 방식은 신선도 요구로 고릅니다

내 테이블의 Silver 계약과 SQL — 약 20분

내 테이블 하나	남길 규칙·초안
필수 1 · 한 행의 의미·PK	어떤 업무 상태를 하나로 묶나요?
필수 2 · 최신 버전·삭제	비교 범위와 삭제 의미·버전 보존
필수 3 · 최신 후보 SQL	PARTITION BY · ORDER BY · rn = 1
확장 · 타입·MERGE 가드	상세 정규화와 새 버전 갱신 조건
확장 · 다섯 기대값	PK·최신·삭제·재실행·무효 입력 거절
확장 · 실행·보류	가능하면 실행 · 부족한 원천 정보 기록

개인 활동 20분 · 필수 세 묶음 12분, SQL·검증 확장 8분

동료 검토 — 과거 변경이 늦게 온다면

동료에게 설명할 것

같은 키를 고른 이유
최신 순서의 근거
삭제 뒤 과거 UPDATE
같은 범위 재실행

검토 뒤 남길 것

확정한 규칙
보완할 SQL·검증
정보가 부족한 보류점

평가 발표보다, 현재 한 일과 선택·막힌 점을 동료에게 나누는 시간입니다

남길 결과와 다음 회차

Silver 계약

한 행·키·순서
삭제·입력 범위

SQL과 검증

정규화·최신 후보
MERGE·예상/실제

다음 5회차

dbt Gold 모델
지표와 품질 테스트

결과 행의 업무 멱등성과 snapshot·파일 쓰기를 구분해 설명합니다